Sesión 3: Agregaciones e Índices en MongoDB

Índice

Operadores de acumulación

Pipeline de agregación con etapas

Teoría de índices (B-tree)

Tipos de índices y su creación

Análisis con explain()

Mejores prácticas (ESR)

¿Qué son los operadores de acumulación?

- Son expresiones especiales utilizadas principalmente dentro de **$group** (etapa de agregación) para calcular valores agregados.
- Permiten operaciones como sumar valores, calcular promedios, encontrar mínimos/máximos, concatenar elementos en arreglos, entre otros, sobre los documentos de cada grupo.
- Actúan sobre los campos de los documentos del grupo actual y producen un resultado agregado por grupo.

Principales operadores de acumulación

- **$sum**: suma de los valores (o conteo de documentos, si se usa $sum: 1).
- **$avg**: calcula el promedio (media) de los valores numéricos.
- **$min** / **$max**: obtiene el valor mínimo o máximo de un campo.
- **$push**: agrega valores a un array (incluye duplicados).
- **$addToSet**: agrega valores a un array solo si no están ya (como conjunto, evita duplicados).
- **$first** / **$last**: toma el primer o último valor de un grupo (tras un sort previo, típicamente).

Ejemplo: uso de acumuladores en $group

Queremos agrupar los productos por `categoria` y calcular:

- `totalProductos`: cantidad de productos en cada categoría.
- `stockTotal`: suma del stock en cada categoría.
- `precioPromedio`: precio medio por categoría.
- `marcas`: lista de marcas distintas presentes en cada categoría.

Usando operadores de acumulación dentro de $group:

```
db.productos.aggregate([
  { $group: {
      _id: '$categoria',
      totalProductos: { $sum: 1 },
      stockTotal: { $sum: '$stock' },
      precioPromedio: { $avg: '$precio' },
      marcas: { $addToSet: '$marca' }
  } }
```

Detalles sobre acumuladores

- **$sum: 1** se usa comúnmente para contar documentos en el grupo.
- **$push** vs **$addToSet**: $push listará todos los valores (permitiendo duplicados), mientras $addToSet solo agrega valores únicos.
- **$first/$last**: Útiles para obtener un valor de un documento al inicio o final de cada grupo, se debe usar en conjunto con un ordenamiento previo ($sort) dentro del pipeline para que tenga sentido.
- Los acumuladores solo están disponibles dentro de contextos de agregación (no se pueden usar en `find` comunes, por ejemplo).

Acumuladores adicionales

- **$stdDevPop / $stdDevSamp**: calculan la desviación estándar poblacional y muestral, respectivamente, de los valores numéricos de cada grupo.
- MongoDB ofrece otros operadores especializados para distintos casos; por ejemplo, $mergeObjects permite combinar subdocumentos de los miembros de un grupo en un solo objeto.
- En versiones recientes existen variaciones como $firstN y $lastN, que obtienen múltiples elementos ordenados de cada grupo (una extensión de $first/$last).
- Es importante revisar la documentación oficial para conocer todos los operadores disponibles y elegir el adecuado según la necesidad.

Resumen: Operadores de acumulación

Operador	Función
$sum	Sumar valores (o contar documentos con 1)
$avg	Calcular el promedio de valores
$min/$max	Valor mínimo / máximo de un campo
$push	Insertar valor en un array (permite duplicados)
$addToSet	Insertar valor en un array, evitando duplicados
$first/$last	Primer/último valor de cada grupo (requiere orden)

¿Qué es la agregación en MongoDB?

- Proceso de cálculo de resultados agregados a partir de colecciones de documentos.
- MongoDB proporciona el **pipeline de agregación**: múltiples etapas (fases) donde la salida de cada etapa es la entrada de la siguiente.
- Permite transformar, filtrar, agrupar y resumir datos de forma potente (similar a `GROUP BY` de SQL pero más flexible).
- Alternativa recomendada frente a Map-Reduce (más compleja) para la mayoría de casos de agregación.

Etapas principales del pipeline

- **$match**: Filtra documentos según una condición (equivalente a `find()` con criterio).
- **$group**: Agrupa documentos por alguna llave y calcula resultados agregados para cada grupo (sumas, promedios, conteos, etc.).
- **$project**: Reestructura cada documento, por ejemplo seleccionando campos o calculando nuevos campos.
- **$sort**: Ordena el resultado según uno o varios campos, ascendente o descendentemente.
- **$limit** y **$skip**: Limitan el número de documentos en la salida o saltan un número de documentos (paginación).
- **$unwind**: Desenrolla arrays, creando un documento separado por cada elemento del array.
- **$lookup**: Combina datos de otra colección (join) dentro del pipeline.

Ejemplo: Pipeline de agregación (I)

Supongamos la colección `productos` con campos `categoria`, `precio`, `stock`, `marca`, etc. Queremos obtener, por categoría:

- El número de productos (**conteo**)
- El stock total disponible (**suma** de `stock`)
- El precio promedio (**promedio** de `precio`)

Podemos construir un pipeline con **$group** (agrupando por `categoria`) después de una etapa opcional de filtro.

Ejemplo: Pipeline de agregación (II)

Primero, si quisiéramos filtrar por categorías específicas (por ejemplo, solo electrónica):

```
pipeline = [
  { $match: { categoria: 'electronica' } },
  { $group: {
      _id: '$categoria',
      totalProductos: { $sum: 1 },
      totalStock: { $sum: '$stock' },
      precioPromedio: { $avg: '$precio' }
    } }
]
db.productos.aggregate(pipeline)
```

En este ejemplo, **$match** filtra los documentos y **$group** calcula por cada categoría el total de productos, la suma de `stock` y el precio medio.

Características del pipeline

- No hay límite fijo en el número de etapas; se pueden combinar libremente según la necesidad.
- Cada etapa maneja hasta 100 MB en memoria. Si una agregación requiere más, se puede usar la opción `allowDiskUse: true` para permitir uso de disco (más lento).
- La salida del pipeline puede enviarse a:
  - El cliente (como cursor de resultados).
  - Otra colección con **$out** (guardando resultados).
- Las agregaciones pueden producir documentos de resultado de hasta 16 MB (límite del tamaño de documento de MongoDB).

$lookup - Uniones entre colecciones

- Operador de agregación que permite realizar uniones entre colecciones, similar a SQL JOIN.
- Sintaxis básica: `{$lookup: {from: 'colecciónAUnir', localField: 'campoLocalObjetivo', foreignField: 'campoDeColeccionAUnirObjetivo', as: 'resultado'}}`

Ejemplo $lookup

Ejemplo práctico: Muestra los nombres de los clientes y el precio del último producto que han comprado.

Resultado: Cada documento tendrá un array con los documentos coincidentes de la otra colección.

```
db.clientes.aggregate([
  {
    $lookup: {
      from: 'pedidos',
      localField: 'ultima_compra',
      foreignField: 'nombre',
      as: 'resultado'
    }
  },{$project:{nombre:1, 'resultado.precio':1}}
])
```

$facet - Múltiples pipelines

- Operador que ejecuta varios pipelines de agregación en una sola operación.
- Sintaxis básica: `{$facet: {nombre1: [etapas], nombre2: [etapas]}}`
- Ideal para dashboards que requieren múltiples métricas en una consulta.
- Cada sub-pipeline procesa los mismos documentos de entrada de forma independiente.

Ejemplo $facet

Ejemplo práctico: Obtener los 5 productos más caros y el total de productos por categoría

```
db.productos.aggregate([
  {    $facet: {
       top_precios: [
         { $sort: { precio: -1 } },
         { $limit: 5 }
       ],
       por_categoria: [
         { $group: {
           _id: "$categoria",
           total: { $sum: 1 }
         }}
       ]}}
])
```

Resumen: Pipeline de agregación

Etapa	Función
$match	Filtrar documentos por un criterio
$group	Agrupar documentos y calcular agregados
$project	Seleccionar o transformar campos
$sort	Ordenar resultados según campos
$limit/$skip	Limitar o paginar resultados
$unwind	Expande documentos por elementos de un array
$lookup	Combinar documentos de otra colección (join)
$out	Escribir resultados en una colección
$facet	Ejecuta varios pipelines de agregación

¿Qué es un índice en MongoDB?

- Un índice es una estructura de datos que mejora la velocidad de las consultas en un campo determinado a costa de espacio adicional y mantenimiento en escrituras.
- Funciona como un índice de un libro: permite encontrar rápidamente los documentos que contienen un valor específico en lugar de revisar cada documento en la colección.
- Sin un índice en un campo, MongoDB debe hacer un **escaneo completo** de la colección (examinar todos los documentos) para encontrar los que cumplen una consulta.

Índices como árboles B (B-Tree)

- MongoDB implementa sus índices utilizando **árboles B** (B-Tree), una estructura de árbol balanceado ampliamente usada en bases de datos.
- En un B-Tree, las llaves (keys) se mantienen ordenadas; esto permite búsquedas binarias eficientes que reducen el número de comparaciones necesarias.
- La búsqueda en un índice B-Tree tiene una complejidad aproximada de $O(\log n)$, comparado con $O(n)$ de una búsqueda sin índice.
- Los nodos del árbol apuntan a hojas que referencian a los documentos reales en la colección, por lo que una vez localizada la clave, se accede rápidamente al documento.

Índice predeterminado y claves únicas

- Cada colección en MongoDB tiene automáticamente un índice B-Tree en el campo `_id` (clave primaria de cada documento). Esto garantiza búsqueda rápida por `_id`.
- Los índices pueden configurarse como **únicos** (unique): imponen que no haya dos documentos con el mismo valor en el campo indexado. El índice _id es único por defecto.
- Índices únicos son útiles para campos que deben ser únicos (como correos electrónicos de usuarios), pero crear uno en un campo con datos duplicados existentes fallará.
- Además del _id, se pueden crear múltiples índices adicionales en otros campos para acelerar diversas consultas (ej. campos frecuentes en filtros u ordenamientos).

Costo y consideraciones de los índices

- Los índices aceleran las consultas de lectura, pero añaden sobrecarga a las operaciones de escritura (insertar/actualizar/eliminar) pues el índice debe mantenerse actualizado.
- Cada índice ocupa espacio en disco y en memoria (mantenerlo en RAM mejora el rendimiento). Se debe asegurar que los índices más utilizados quepan en la memoria disponible.
- Demasiados índices en una colección pueden degradar el rendimiento de escritura. Es recomendable indexar solo lo necesario (ver Mejores Prácticas).
- Se puede revisar el uso de índices con herramientas como `explain()` o estadísticas de índice para identificar índices no utilizados.

Limitaciones de los índices

- **Cantidad**: Cada colección puede tener hasta 64 índices distintos como máximo. Un número excesivo de índices puede indicar sobreindexación.
- **Campos por índice**: Un índice compuesto puede incluir hasta 32 campos. Mientras más campos, más específico pero también más costoso de mantener.
- **Tamaño de clave**: La longitud máxima de una clave indexada es 1024 bytes. Si el valor de un campo excede este tamaño, no podrá indexarse completamente (esas entradas se omiten).
- **Índices especiales**: Existen índices geoespaciales (`2d`, `2dsphere`) para datos de coordenadas, e índices *hashed* para distribuir datos uniformemente (usados en sharding).

Resumen: Conceptos de índices

Concepto	Descripción
B-Tree	Estructura de árbol balanceado utilizada para índices
_id por defecto	Cada colección tiene índice único en _id automáticamente
Búsqueda $O(\log n)$	Búsqueda mediante índice (tiempo logarítmico)
Escaneo completo	Sin índice, la consulta recorre todos los documentos ($O(n)$)
Costo en escrituras	Cada índice adicional ralentiza inserciones y actualizaciones
Tamaño de índice	Los índices usan espacio en disco y memoria

Creación de índices en MongoDB

- Comando principal: `db.<colección>.createIndex({ campo: orden }, { opciones })`. El *orden* es 1 para ascendente o -1 para descendente.
- Ejemplo: `db.productos.createIndex({ categoria: 1 })` crea un índice ascendente en el campo `categoria`.
- Para crear un índice **único**: añadir `unique: true` en opciones. Ej: `db.usuarios.createIndex({ email: 1 }, { unique: true })`.
- MongoDB permite múltiples índices por colección. Se deben usar los nombres de campos y orden para especificarlos. El mismo campo puede aparecer en varios índices (p.ej., solo vs compuesto) pero no es recomendable duplicar índices innecesariamente.

Índices de campo único

- Índice más básico: sobre un solo campo. Acelera búsquedas que filtran por ese campo.
- Ejemplo: índice en `categoria` permite acelerar consultas `find({ categoria: 'electronica'})`.
- Se puede crear índice en cualquier campo, incluyendo campos anidados usando la notación de punto. Ej: `'direccion.ciudad': 1`.
- Limitación: solo optimiza consultas por ese campo exacto, no por combinaciones con otros campos (para ello usar un índice compuesto).

Índices compuestos

- Un índice compuesto incluye más de un campo. Útil para consultas que utilicen múltiples campos en filtros y/o ordenamientos.
- Ejemplo: `db.productos.createIndex({ categoria: 1, precio: -1 })` crea un índice primero por `categoria` (asc), luego `precio` (desc).
- El orden de los campos en el índice es crítico: el índice anterior puede usarse para buscar por categoria sola o categoria+precio, pero no eficientemente solo por precio.
- Un buen diseño de índice compuesto sigue la regla ESR (Igualdad, Orden, Rango) para ordenar los campos (ver Mejores Prácticas).

Índices multiclave (arrays)

- Si un campo indexado es un **array**, MongoDB indexa automáticamente cada elemento del array (cada valor del array actúa como una entrada separada en el índice). A estos índices se les llama *multiclave*.
- Permiten consultas eficientes dentro de arrays. Ejemplo: si `tags` es un array de etiquetas y está indexado, `find({ tags: 'promo' })` usará el índice para encontrar documentos donde cualquier elemento del array sea 'promo'.
- Limitación: no se permite más de un campo de tipo array en un mismo índice compuesto (MongoDB no puede indexar dos arrays a la vez en un solo índice).
- Los índices multiclave pueden aumentar el tamaño del índice ya que un solo documento puede generar múltiples entradas (una por cada elemento del array).

Índices de texto

- Especial para búsqueda de texto. Permiten realizar búsquedas por palabras clave en campos de texto (búsqueda 'textual', no exacta).
- Creación: `db.articulos.createIndex({ descripcion: 'text' })` indexa el campo `descripcion` para búsqueda de texto.
- Una colección puede tener a lo sumo un índice de texto a la vez (que puede incluir varios campos combinados).
- Consultas: usan $text en `find`. Ej: `db.articulos.find({ $text: { $search: 'palabra clave' } })` encuentra documentos cuyo texto coincida con la búsqueda.
- Idioma: se puede especificar el idioma para ajustes de stemming y stop words. Ej: `{ default_language: 'spanish' }` al crear el índice (por defecto es inglés).

Índices TTL y parciales

- **Índice TTL (Time To Live)**: índice especial que elimina automáticamente documentos tras cierto tiempo.
  - Se crea en campos de fecha. Ej: `db.logs.createIndex({ fecha: 1 }, { expireAfterSeconds: 3600 })` eliminará documentos una hora después de la fecha indicada.
  - Útil para datos temporales (logs, sesiones) que expiran.
- **Índice parcial**: indexa solo documentos que cumplen un filtro dado.
  - Ej: `db.personas.createIndex({ edad: 1 }, { partialFilterExpression: { edad: { $gte: 18 } } })` indexa solo documentos con `edad >= 18`.
  - Reduce el tamaño del índice y mejora rendimiento cuando solo ciertos documentos son relevantes para consultas.
  - Reemplaza en gran medida la funcionalidad de índices **sparse** (dispersos) de versiones antiguas, que excluían documentos sin el campo.

Resumen: Tipos de índices

Tipo de índice	**Uso principal**
Campo único	Indexar un campo individual para búsquedas directas
Compuesto	Indexar combinación de campos (consultas por varios campos)
Multiclave	Indexar campos que contienen arrays (cada elemento indexado)
Texto	Búsqueda por palabras clave en campos textuales
TTL	Eliminación automática de documentos antiguos
Parcial	Indexar solo un subconjunto de documentos que cumplen condición
Único	Garantizar unicidad de valores en el campo indexado

¿Para qué sirve explain()?

- La función `explain()` permite analizar cómo el motor de MongoDB ejecuta una consulta u operación (find, aggregate, update, etc.).
- Provee información sobre el **plan de ejecución**: si usa índices o hace escaneo completo, cuántos documentos examinó, cuánto tardó, etc.
- Es una herramienta esencial para optimizar el rendimiento de las consultas, ayudando a identificar cuán eficiente es una consulta y si necesita índices adicionales o reescritura.

Uso básico de explain()

- Modo común: encadenar `.explain()` a una consulta. Ej: `db.productos.find({ categoria: 'electronica' }).explain('executionStats')`.
- `explain()` puede usarse con diferentes operaciones: find, aggregate, count, update (en update/delete se antepone `db.collection.explain().update(...)`).
- **Modos de verborosidad**: `'queryPlanner'` (por defecto, solo plan), `'executionStats'` (plan + estadísticas de ejecución), `'allPlansExecution'` (estadísticas de todos los planes considerados).
- Para análisis de rendimiento se suele usar `'executionStats'` para obtener el número real de documentos examinados y el tiempo empleado.

Interpretando la salida de explain()

Al ejecutar `explain('executionStats')`, la salida es un documento JSON con mucha información. Componentes clave:

- **queryPlanner**: Detalles del plan escogido (y alternativas si las hubo).
- **command**: La consulta u operación analizada.
- **winningPlan**: El plan de ejecución seleccionado por el optimizador.
  - Dentro de este, `stage` indica la etapa principal (p. ej. `COLLSCAN` para escaneo completo o `IXSCAN` si usó un índice).
  - Si es `IXSCAN`, habrá detalles como `indexName` (nombre del índice usado) e `indexBounds` (rangos de búsqueda en el índice).
- **executionStats**: Estadísticas posteriores a la ejecución:
  - `nReturned`: documentos devueltos por la consulta.
  - `totalDocsExamined`: total de documentos examinados (debería ser igual a nReturned si usa un índice muy selectivo; mucho mayor si fue COLLSCAN).
  - `executionTimeMillis`: tiempo que tardó la operación en milisegundos.

Ejemplo: índice vs. escaneo completo

- Consulta: `db.productos.find({ precio: 100 }).explain('executionStats')`.
- Sin índice en `precio`: `stage: 'COLLSCAN'`, `totalDocsExamined` = N (todos los documentos de `productos`), `nReturned` = M (los que tienen precio = 100).
- Si creamos un índice en `precio` y repetimos la consulta: `stage: 'IXSCAN'` usando índice `precio_1`, `totalDocsExamined` = M (solo examina los docs con precio=100), `nReturned` = M.
- Diferencia: el índice evitó examinar N-M documentos irrelevantes, reduciendo drásticamente el trabajo y el tiempo de la consulta.
- **Conclusión**: `explain()` demuestra el beneficio del índice mostrando el cambio de COLLSCAN a IXSCAN y un menor `totalDocsExamined`.

Optimización basada en explain()

- Use `explain()` para identificar cuellos de botella:
  - Si ve `COLLSCAN` donde esperaba `IXSCAN`, considere crear un índice para ese filtro.
  - Revise `totalDocsExamined` vs `nReturned`: si `totalDocsExamined` $\gg$ `nReturned`, la consulta está leyendo demasiados documentos.
- Asegúrese de que las consultas aprovechen los índices existentes:
  - A veces se debe reescribir la consulta para que use el índice (ej: usar el mismo orden de campos que el índice compuesto, combinar condiciones en una sola consulta en lugar de varias).
  - Otras veces, eliminar índices redundantes ayuda al optimizador a elegir el plan correcto.
- **Monitorización continua**: en aplicaciones críticas, monitorear los planes de consulta periódicamente (o usar comandos de perfilado) ayuda a mantener el rendimiento óptimo.

Resumen: Modos de explain

Modo	Descripción
`'queryPlanner'`	Muestra el plan de ejecución seleccionado (sin ejecutar la consulta
`'executionStats'`	Ejecuta la consulta y provee estadísticas de ejecución (tiempo, doc
`'allPlansExecution'`	Como executionStats, pero incluye detalles de todos los planes con

Importancia de un buen diseño de índices

- Índices bien diseñados aseguran que las consultas comunes sean eficientes, reduciendo la latencia y la carga de CPU/IO.
- Un mal diseño (ej: índices faltantes o en orden incorrecto) puede provocar que incluso bases de datos pequeñas tengan bajo rendimiento.
- La **regla ESR** es una guía fundamental para construir índices compuestos eficaces.
- Se deben escoger índices basados en los **patrones de consulta** de la aplicación: qué campos se filtran, ordenan o combinan con más frecuencia. Para comprobar estos campos, se debe utilizar esta sentencia:

```
db.nombreColeccion.aggregate( [ { $indexStats: {} } ] )
```

Regla ESR: Equality, Sort, Range

- ESR (Igualdad, Orden, Rango) sugiere el orden de los campos en un índice compuesto:
  - Primero, los campos por los que se consulta con **igualdad** (==, valor exacto).
  - Luego, los campos por los que se realiza **ordenamiento** (sort) en la consulta.
  - Finalmente, los campos que se consultan por **rango** ($gt, $lt, $in, etc.).
- Siguiendo este orden, el índice podrá soportar eficientemente las tres partes de la consulta:
  - Las búsquedas por igualdad aprovechan la parte inicial del índice.
  - Con el índice ya ordenado según el siguiente campo, la consulta puede evitar un sort en memoria.
  - Finalmente, los filtros por rango se aplican sobre el subconjunto ya filtrado y ordenado.

Ejemplo de aplicación de ESR

Consulta frecuente: Obtener usuarios **activos** (`estado: 'activo'`) de cierta **edad mínima** (`edad > 30`) ordenados por **apellido**.

- Filtro de igualdad: `estado = 'activo'`.
- Filtro de rango: `edad > 30`.
- Ordenamiento: por `apellido` ascendente.

Según ESR, un índice ideal sería:

```
db.usuarios.createIndex(
  { estado: 1, apellido: 1, edad: 1 }
)
```

- `estado` (igualdad) primero, `apellido` (sort) segundo, `edad` (rango) al final.
- Este índice permitirá buscar rápidamente usuarios activos, ya ordenados por apellido, y luego filtrar por el rango de edad.

Selectividad y composición de índices

- **Selectividad**: Un índice es más útil cuando filtra una fracción pequeña de la colección. Campos con valores muy repetidos (ej: campo booleano) no son buenos candidatos, ya que el índice no reduce mucho la búsqueda.
- **Índices compuestos vs. múltiples índices**: En lugar de crear muchos índices simples para distintos campos, a menudo conviene un índice compuesto que abarque varios campos usados juntos en consultas.
- Un índice compuesto puede servir para consultas por el primer campo solo, o por el primer y segundo juntos, etc. En cambio, múltiples índices separados no pueden combinarse fácilmente en un solo plan eficiente (MongoDB suele usar solo un índice por consulta; la intersección de índices es limitada).
- Revisa las consultas más pesadas con `explain()` y ajusta los índices en consecuencia.

Evitar sobreindexación

- **Índices innecesarios**: Tener índices que no se usan en consultas reales solo añade sobrecarga en escrituras y consume memoria. Monitorea el uso de índices (`indexStats`, `usageStats`) y elimina los que no sean útiles.
- **Índices redundantes**: Evita índices duplicados o que sean prefijo de otros. Ejemplo: si existe índice {estado:1, apellido:1}, un índice adicional solo {estado:1} es redundante porque las consultas por estado pueden usar el índice compuesto.
- **Demasiados índices**: Cada índice extra ralentiza inserciones/actualizaciones. Prioriza los índices que benefician consultas críticas. En general, mantén un equilibrio según las necesidades de lectura vs. escritura de tu aplicación.
- Reevalúa periódicamente la estrategia de índices conforme cambian los patrones de acceso a los datos.